

Référence des opérations PDF · PDF Equilibrist

< · [Index](#) · [Architecture](#) · [Contributing](#) · [Build](#)

Les fonctions de operations/ sont sans UI et reçoivent directement un fitz.Document. Elles modifient le document en place (sauf indication contraire).
Le tab appelant émet document.changed après chaque appel.

▪ operations/edit.py

`extract_text_blocks(doc, page_index) · list[TextBlock]`

Extrait tous les spans de texte d'une page avec leurs métadonnées de style.

Champ <code>TextBlock</code>	Type	Description
<code>block_id</code>	int	Identifiant unique sur la page
<code>rect</code>	fitz.Rect	Rectangle englobant du span
<code>origin</code>	fitz.Point	Point baseline pour <code>insert_text()</code>
<code>text</code>	str	Contenu textuel actuel
<code>fontsize</code>	float	Taille en points PDF
<code>color</code>	tuple(r,g,b)	Couleur normalisée 0-1
<code>fontname</code>	str	Nom de la police d'origine
<code>font_buffer</code>	bytes\None	Police embarquée (pour réinsertion fidèle)

> · Retourne [] si la page est une image (PDF scanné sans couche texte).

`apply_text_edits(doc, page_index, original_blocks, edited_texts)`

Applique les modifications textuelles sur une page en deux passes :

1. Redaction : `add_redact_annot(rect, fill=None)` + `apply_redactions()`
· supprime le texte original sans laisser de rectangle blanc
2. Réinsertion : `insert_text(origin, new_text, ...)` au point baseline exact

Seuls les blocs dont le texte a changé (`edited_texts[bid] != original_blocks[bid].text`) sont traités.

Paramètres :

- `original_blocks` : liste issue de `extract_text_blocks()`
- `edited_texts` : {`block_id`: nouveau_texte} · seuls les blocs modifiés

`compress(doc, save_path)`

Sauvegarde avec compression maximale (`garbage=4`, `deflate=True`).
Ne modifie pas le document en mémoire · écrit un nouveau fichier.

``add_watermark(doc, text, color=(0.6,0.6,0.6))``

Insère un texte diagonal (45°, fontsize=48) centré sur chaque page.
Le centrage est approximatif (13 pts/caractère).

``add_text(doc, page_index, text, point, fontsize=12, color=(0,0,0))``

Insère un texte libre au point point (coordonnées PDF, point baseline).

``add_image(doc, page_index, image_path, rect)``

Insère une image dans le rectangle rect (étiré pour remplir).

■ operations/pages.py

``rotate_pages(doc, angle, page_indices=None)``

Rotation additive (rotation + angle) % 360 sur les pages spécifiées.
Angles valides : +90 (droite), -90 (gauche), 180.

``invert_pages(doc) · fitz.Document``

Retourne un nouveau document avec pages en ordre inverse.
L'original n'est pas modifié.
Le tab appelant doit fermer l'ancien et assigner le nouveau avant d'émettre changed.

``split_pdf(doc, output_dir) · list[Path]``

Un PDF par page dans output_dir. Retourne les chemins créés.

``merge_pdfs(paths) · fitz.Document``

Concatène plusieurs PDFs en un nouveau document en mémoire (non sauvegardé).

``extract_pages(doc, page_indices) · fitz.Document``

Retourne un nouveau document ne contenant que les pages demandées (copiées dans l'ordre croissant de leur index d'origine, quel que soit l'ordre de page_indices). Le document source n'est pas modifié.

``insert_page(doc, after_index, src_path=None)``

- src_path=None · insère une page blanche (A4 par défaut)
- src_path fourni · insère toutes les pages du PDF source

``crop_page(doc, page_index, rect)``

Définit la CropBox (zone visible) · non destructif, le contenu reste dans le PDF.

```
`set_page_size(doc, page_index, width, height)`
```

Modifie la MediaBox. Dimensions standard en points PDF :

- A4 portrait : 595 × 842
- A4 paysage : 842 × 595
- Letter : 612 × 792

■ **operations/annotate.py**

Toutes les fonctions retournent l'objet fitz.Annot créé.

Les annotations sont des objets PDF distincts du contenu de la page.

```
`highlight(page, quads)`
```

Surlignage jaune. quads = [fitz.Rect(bbox).quad for word in words].

```
`strikeout(page, quads)`
```

Biffure (ligne horizontale au centre du texte).

```
`underline(page, quads)`
```

Soulignement sous le texte.

```
`add_text_box(page, rect, text, fontsize=11, color=(0,0,0))`
```

Zone de texte FreeText avec fond jaune pâle et bordure dorée.

```
`add_stamp(page, rect, stamp_id=0)`
```

Tampon prédéfini PDF (0=Draft, 1=Approved, 2=Experimental·).

■ **operations/convert.py**

```
`detect_office_engine() · 'msoffice' | 'libreoffice' | None`
```

Détecte le moteur disponible :

1. MS Office · registre HKLM\SOFTWARE\Microsoft\...\App Paths\WINWORD.EXE
2. LibreOffice · chemins standards + shutil.which("soffice")

```
`office_to_pdf(input_path, output_path) · Path`
```

Convertit un fichier Office (.docx/.xlsx/.pptx) en PDF.

Moteur	Mécanisme
MS Office	<code>`docx2pdf.convert()`</code> (automation COM)
LibreOffice	<code>`soffice --headless --convert-to pdf`</code>

Lève RuntimeError si aucun moteur n'est disponible.

```
`to_images(doc, output_dir, zoom=2.0, fmt='png') · list[Path]`
```

Un fichier image par page. Zoom recommandé : 2.0 (144 dpi).

``to_word(doc, src_path, output_path) · Path``

Utilise pdf2docx.Converter. Nécessite le chemin fichier (pas le fitz.Document).

``to_excel(doc, output_path) · Path``

Extrait les tableaux via pdfplumber. Une feuille par page.

Efficace uniquement sur les PDF avec des tableaux à bordures visibles.

``to_powerpoint(doc, output_path, zoom=1.5) · Path``

Une slide par page (image rasterisée). Le texte n'est pas éditable dans PowerPoint.

``image_to_pdf(image_paths, output_path) · Path``

Chaque image · une page A4 (595×842 pts). L'image est étirée pour remplir la page.

■ operations/protect.py

``encrypt(doc, save_path, user_password, owner_password=")``

Chiffrement AES-256 dans un nouveau fichier. Permissions accordées :

PDF_PERM_PRINT | PDF_PERM_COPY.

Le document source reste inchangé en mémoire.

``decrypt(doc, password) · bool``

Appelle doc.authenticate(password). Retourne True si succès.

Le déchiffrement est en place (déverrouille en mémoire, sans créer de fichier).

``is_encrypted(doc) · bool``

Vérifie si le document nécessite un mot de passe.

■ ui/print_dialog.py

``print_document(document, parent=None) · bool``

Ouvre la boîte d'impression Windows et envoie le document à l'imprimante.

Étape	Détail
QPrinter	HighResolution · utilise le DPI réel de
QPrintDialog	Boîte système : sélection imprimante, pl
Zoom	`dpi / 72` · 1 pt PDF = 1/72 pouce, indé
Rendu	`fitz.Page.get_pixmap(matrix)` à la réso
Envoi	`QPainter.drawPixmap(page_rect, pixmap)`

Retourne True si imprimé, False si annulé ou document fermé.

Raccourci : Ctrl+P · également accessible via Fichier · Imprimer et bouton ribbon.